

Association-Rule Mining with Apriori on Online Retail: A Browser-Based Reproducibility Study

Student: Polina Ivanilova **Email:** pvivanilova@edu.hse.ru **Team:** Individual **Date:** 2026-10-08

Assignment: Association Rules (Week 4)

Abstract

Online stores often show the "frequently bought together" block, but ranking by raw co-occurrence pushes merely popular items to the top. I implemented an Apriori miner directly in the browser over 17,080 retail baskets and ranked rules by support, confidence, and lift. At support $\geq 1\%$ and confidence $\geq 30\%$ it produced 950 rules, including a china-pattern pair with confidence 78.29% and lift 17.1440. Confidence alone is not enough — lift is the more reliable criterion for choosing really interesting rules.

Index Terms — association rules, Apriori, lift, market-basket analysis

1. Introduction

Problem statement. Mine "baskets with A also contain B" rules at support, confidence, and lift thresholds, keeping only lift > 1 . One problem with using confidence alone is that it does not account for how common the consequent is. This was especially visible for rules pointing to popular items. I also observed that lowering the support threshold can increase the number of rules very quickly. Finally, the mined relationships describe co-occurrence and should not be interpreted as causal relationships.

Motivation. Fourth assignment of an AI-assisted Recommender Systems course, moving from Week 3 personalised similarity to the co-occurrence signal. Association rules power "frequently bought together" widgets with no user model: the non-personalised baseline every shop needs before personalisation.

Concrete example. With the default settings of 1% support and 30% confidence, the application returns 950 rules. For example, GREEN $\rightarrow$ ROSES REGENCY TEACUP has 78.29% confidence and a lift of 17.1440. The reverse rule has a lower confidence of 69.36%, while the lift remains the same.

Contributions.

- Browser Apriori app implementing the seven TODO(hw4) stubs (baskets, counts, three metrics, miner, bidirectional rules) with a sortable results table; no runtime framework.
- Verified numbers — 11/11 harness, a hand-checked rule, a 1,200-row independent cross-check with 0 mismatches — plus the required useful/misleading, threshold, direction, and validation analyses.

2. Related Work

Prior work consulted: the UCI Online Retail data and case study [1] [2], the Apriori papers [3] [4], and association-rule background [5].

Alternatives considered. *FP-growth* — faster at 0.5% support, rejected: the brief requires Apriori and posting-list intersection is already interactive. *Confidence-only ranking* — rejected: Results (B) clears 30% confidence while sitting at the lift floor. *Week-3 collaborative filtering* — needs user history; rules serve anonymous shoppers.

3. Method

Approach. Baskets are InvoiceNo groups from UCI Online Retail (id 352, Dec 2010–Dec 2011) [2] after the starter cleaning (no cancellations, positive quantities and prices, no guests or non-products): 17,080 baskets, 3,653 items, no per-basket timestamps. dedupeBasket collapses repeats in first-appearance order so support means share of baskets; metrics are $\text{support} = \text{count}(A \cup B)/N$, $\text{confidence} = \text{count}(A \cup B)/\text{count}(A)$, $\text{lift} = \text{confidence}/(\text{count}(B)/N)$, each returning {value, defined} with defined=false on zero denominators. Top item 85123A (WHITE HANGING HEART) sits in 1,959 baskets (11.47%); nothing exceeds ~11.5%.

Pipeline. (1) dedupeBasket builds the basket index. (2) countItemset intersects sorted posting lists. (3) findFrequentItemsets runs level-wise Apriori, pruning candidates with any infrequent (k-1)-subset [3] [4]. (4) generateRules splits each itemset both ways (bitmask over non-empty proper subsets), filters by confidence, and lifts against global N. (5) The page applies the slider thresholds, renders the sortable table, and opens a detail panel with Reverse direction per rule.

Tools & libraries.

- HTML5 / CSS3 / vanilla JavaScript (ES6), no runtime framework — docs: <https://developer.mozilla.org/>
- Python 3.11.9, local cross-check script only (not shipped) — <https://www.python.org/>
- Microsoft Edge (Chromium) headless --print-to-pdf for the submitted PDF — <https://www.microsoft.com/edge>
- Git / GitHub CLI for version control and Pages deploy — <https://cli.github.com/>

Key design decisions. *Sets, not multisets:* a quantity-30 line counts once, matching "share of baskets". *Both directions with global-N lift:* confidences differ while lifts are provably identical. *Thresholds 1%/30% + lift > 1:* I chose 1% support and 30% confidence as the default thresholds. At this support level, an itemset has to occur in roughly 171 baskets, which removes many rare combinations while still keeping common product groups. The 30% confidence threshold further reduces the number of rules. The lift > 1 condition does not actually remove any rules at these settings, but I kept it because it becomes useful when lower confidence thresholds are tested.

Configuration. Support slider 0.5–10% (default 1.0%), confidence slider 10–100% (default 30%); deterministic code, no random seeds; runs in any modern browser; repo PollyIva/Association-Rules-Recommender, branch main.

4. Experiments

Setup. Final table at 1%/30% with lift > 1 retained. Extra runs at 0.5%/30%, 3%/30%, 1%/60% (plus 1%/90% and 3%/60% for the sweep). Full exports saved for the 950-, 12-, and 238-row settings; the 10,723-row 0.5%/30% run is cited by status-line count. Screenshots of both analysed rules are saved in useful rule/ and weak rule/.

Worked example. (assignment §6.7 hand-check): 23171 → 23172 with counts 269/224/202 and N = 17,080 gives support $202/17080 = 1.18\%$, confidence $202/269 = 75.09\%$, lift $0.750929/(224/17080) = 57.2584$ — exactly what the page shows; reversed confidence is $202/224 = 90.18\%$ at identical lift.

Results (A): useful rule (spec §4.2 item 1). GREEN → ROSES REGENCY TEACUP: 541/691/780, support 3.17%, confidence 78.29%, lift **17.1440** (reverse 69.36%). The rule is interesting because the consequent appears in only 4.57% of all baskets, while the conditional probability is 78.29%. Customers

who buy the green item are therefore much more likely to buy the ROSES teacup than a random customer. This makes the rule a reasonable candidate for a "Complete the set" or bundle recommendation.

Controls

Minimum support 1.0%

0.5% – 10% of baskets.

Minimum confidence 30%

10% – 100%.

Run rules Run tests

Done — 950 rule(s) at support $\geq$ 1.0% and confidence $\geq$ 30%.

Test log

Press "Run tests" to execute the self-checks.

Rules

950 rules.

Antecedent (A)	Consequent (B)	count(A $\cup$ B)	count(A)	count(B)	support	confidence	lift
22386 — JUMBO BAG PINK POLKADOT	850998 — JUMBO BAG RED RETROSPOT	546	869	1584	3.20%	62.83%	6.7749
850998 — JUMBO BAG RED RETROSPOT	22386 — JUMBO BAG PINK POLKADOT	546	1584	869	3.20%	34.47%	6.7749
22697 — GREEN REGENCY TEACUP AND SAUCER	22699 — ROSES REGENCY TEACUP AND SAUCER	541	691	780	3.17%	78.29%	17.1440
22699 — ROSES REGENCY TEACUP AND SAUCER	22697 — GREEN REGENCY TEACUP AND SAUCER	541	780	691	3.17%	69.36%	17.1440
22726 — ALARM CLOCK BAKELIKE	22727 — ALARM CLOCK BAKELIKE	530	784	877	3.10%	67.60%	13.1658

Fig. — 1%/30% run: teacup rule in the results table

Selected rule

Antecedent (A) 22699 — ROSES REGENCY TEACUP AND SAUCER

Consequent (B) 22697 — GREEN REGENCY TEACUP AND SAUCER

count(A $\cup$ B) 541

count(A) 780

count(B) 691

support 3.17%

confidence 69.36%

lift 17.1440

Reverse direction (B $\rightarrow$ A): confidence 78.29%, lift 17.1440. Confidence changes with direction; lift does not.

Reverse direction (B $\rightarrow$ A)

Fig. — Teacup rule detail, reversed: 69.36% confidence, lift 17.1440

Results (B): misleading rule (spec §4.2 item 2). HOME BUILDING BLOCK WORD $\rightarrow$ 85123A: 209/690/1959, support 1.22%, confidence 30.29%, lift **2.6409** (second-lowest; minimum 2.6366). This rule shows why confidence alone is not enough. 85123A already occurs in 11.47% of all baskets, so a confidence of about 30% does not indicate a particularly strong association. The lift of 2.6409 is also close to the minimum observed at these thresholds. Therefore, this rule is much less convincing than the teacup example despite passing the same confidence threshold.

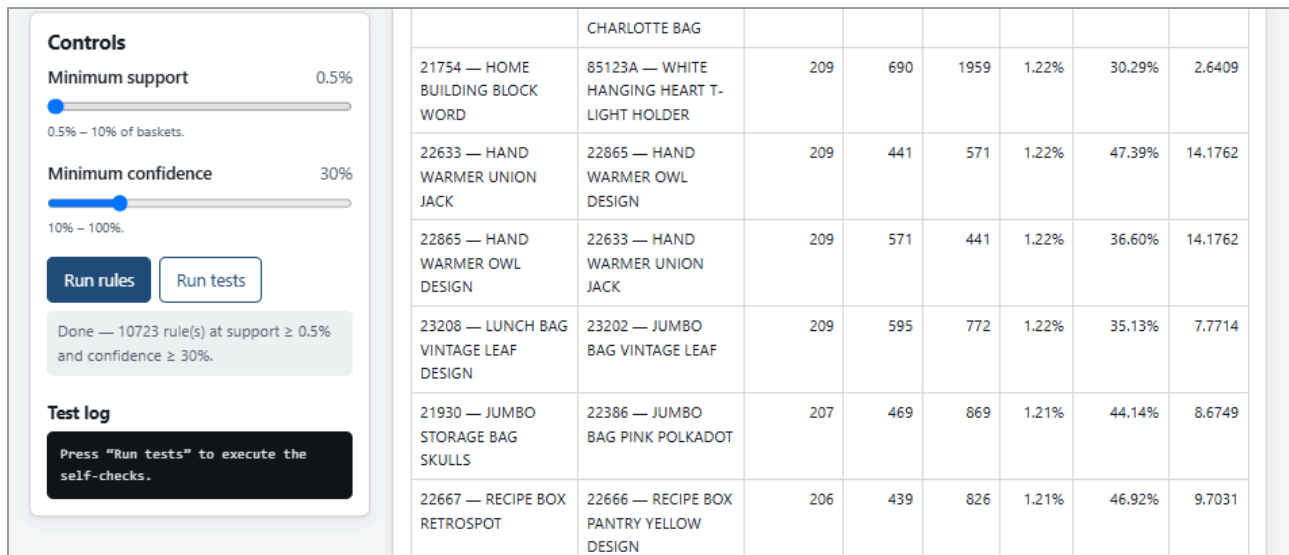


Fig. — 0.5%/30% run: weak rule in the results table

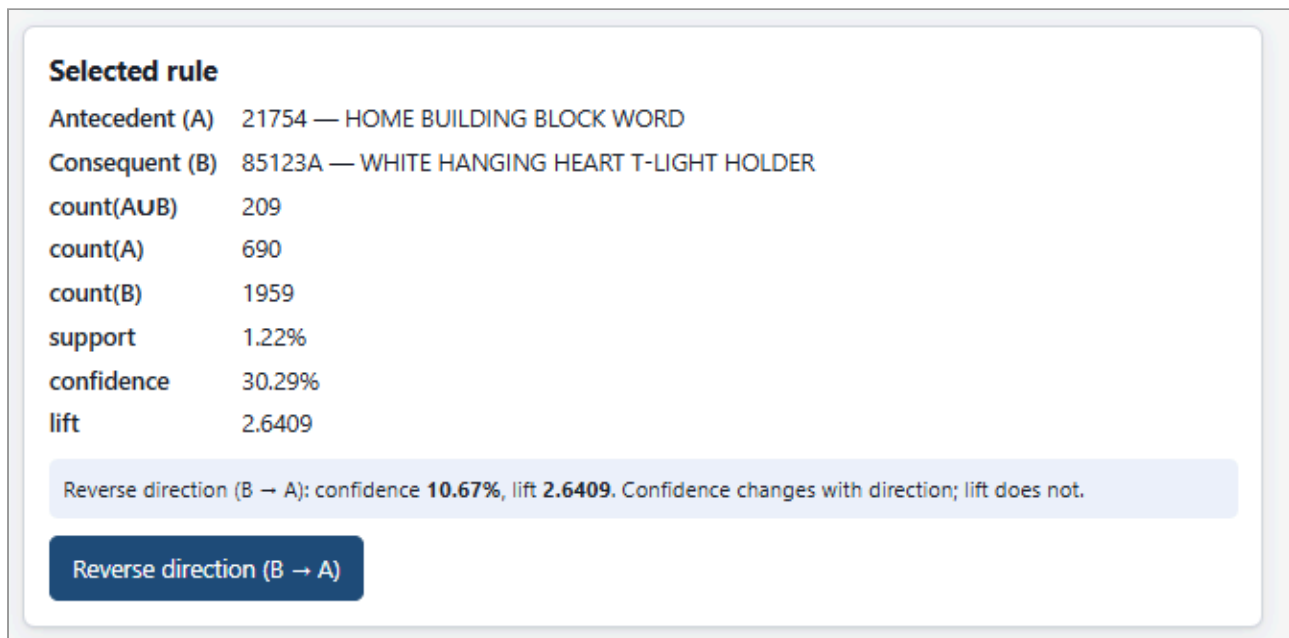


Fig. — Weak rule detail: 209/690/1959, 30.29% confidence, lift 2.6409

Results (C): thresholds (spec §4.2 item 3).

Support	Confidence	Itemsets	Rules
0.5%	30%	5,255	10,723
1%	30%	1,219	950
3%	30%	115	12
1%	60%	1,219	238
1%	90%	1,219	18
3%	60%	115	5

The support threshold had a much stronger effect on the number of mined rules. Reducing support from 1% to 0.5% increased the number of rules from 950 to 10,723. In contrast, changing confidence from 30% to 60% or 90% did not change the number of frequent itemsets; it only reduced the number of rules generated from them. This shows that support mainly controls the size of the search space, while confidence filters the resulting rules.

Direction (optional). Confidences differ whenever counts differ (teacups 78.29 ↔ 69.36; tea plates 75.09 ↔ 90.18; jumbo bags 62.83 ↔ 34.47) while lifts stay identical (17.1440, 57.2584, 6.7749). The direction therefore matters when deciding which item should trigger the recommendation, while lift remains the same for both directions.

Comparison vs. baseline. I used lift = 1 as the independence baseline. At the default 1%/30% settings, the lift filter does not remove any rules because the minimum possible lift for a rule with 30% confidence and consequent 85123A is already about 2.62. This means that the lift > 1 condition is redundant for this particular configuration. However, the additional experiment with a 10% confidence threshold showed why the condition can still be useful: the page admits CAKESTAND → WHITE HEART at 13.92% confidence with lift 1.2137, which is only slightly above independence. An offline 0.5%/0.5% re-mine also found 12 of 21,026 rules with lift < 1 (minimum 0.9267).

Verification. Harness 11/11 (baseline was 2 passed / 9 pending); hand-check matches in both directions; all 1,200 exported rows re-derived from raw counts with 0 mismatches and exact header counts; exports plus four screenshots saved.

5. Discussion

Failure case. A draft called the BLOCK-WORD rule (lift 2.6409) the table minimum; the exhaustive cross-check sort printed a lower row first:

```
23321 SMALL WHITE HEART OF WICKER → 85123A ... lift 2.6366
```

Root cause. The claim came from looking at rows near the bottom of the table instead of sorting all 950 rules. The bottom region is dominated by rules into 85123A without being ordered among themselves.

Fix + verification. *Detection:* the sorted cross-check output above. *Change:* corrected to second-lowest with both numbers stated. *Confirmation:* re-sorted the full export — the five lowest lifts all point at 85123A and all 950 exceed 1.

What worked.

- The posting-list implementation worked well for this dataset: 5,255 frequent itemsets could be mined directly in the browser without an external data-mining library.
- Showing lift together with the consequent base rate helped distinguish a genuinely strong association from a rule that mainly benefits from a popular consequent.
- Export-based evidence made the reported numbers traceable and helped identify the error described above.

What surprised me.

- Not a lot of items with lift < 3

- A 2× threshold loosening multiplies rules more than 10× — the sliders act at different pipeline stages with different units of effect.

Next improvement. As a next step, I would use the transaction dates to split the data into training and validation periods, mining rules on the earlier period and keeping only those whose lift stays above 1 on later transactions. For the strongest rules, such as the teacup example, an A/B test would be needed before using them in an actual recommendation widget.

6. AI Usage Disclosure

AI tools used. OpenCode (AI coding CLI, model big-pickle and Muse Spark 1.3 at the end (big-pickle stop working)): code generation, debugging, cross-check assembly, documentation.

What I personally verified.

- Ran the required settings in the UI: 1%/30%, 3%/30%, 1%/60%, and 0.5%/30%.
- Checked and saved the tables used in the report.
- Checked the selected rules and their reverse-direction views.
- Checked the lowest-lift rules in the exported results.
- Reviewed the report and rewrote some parts.

What I trusted without verification. code & algorithms realisation.

References

- [1] D. Chen, S. L. Sain, and K. Guo, "Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining," *J. Database Mark. Cust. Strateg. Manag.*, vol. 19, no. 3, 2012. [Online]. Available: <https://archive.ics.uci.edu/dataset/352/online+retail>. [Accessed: 2026-10-08]. (As listed on the live UCI page [2]; the assignment sheet lists *J. Cases Inf. Technol.* — we follow the verifiable source.)
- [2] D. Chen, "Online Retail [Dataset]," UCI Machine Learning Repository, 2015. [Online]. Available: <https://archive.ics.uci.edu/dataset/352/online+retail>. [Accessed: 2026-10-08].
- [3] R. Agrawal, T. Imieliński, and A. Swami, "Mining association rules between sets of items in large databases," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, Washington, DC, USA, 1993, pp. 207–216. [Online]. Available: <https://doi.org/10.1145/170035.170072>. [Accessed: 2026-10-08].
- [4] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules," in *Proc. 20th Int. Conf. Very Large Data Bases (VLDB)*, Santiago, Chile, 1994, pp. 487–499. [Online]. Available: <https://www.vldb.org/conf/1994/P487.PDF>. [Accessed: 2026-10-08].
- [5] Wikipedia contributors, "Association rule learning," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Association_rule_learning. [Accessed: 2026-10-08].